AI Practical -1

Name: Sagar Wavhal
Roll No: 81
Class & Div: TE - B

Title: Implement DFS and BFS Algorithm. Use and Undirected Graph and develop a Recursive Algorithm for searching all the vertices of the graph or tree data structure.

Program: Breadth First Search (BFS):

```
# Graph definition
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}

# List to keep track of visited nodes
visited = []

# Initialize a queue
queue = []

# BFS function
def bfs(visited, graph, node):
   visited.append(node)
   queue.append(node)

   while queue:
      s = queue.pop(0)
      print(s, end=" ")

      for neighbour in graph[s]:
         if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)
```

```
# Driver Code
bfs(visited, graph, 'A')
```

OUTPUT:
A B C D E F

2. Depth-first Search:

```
# Using a Python dictionary to act as an adjacency list
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}

# Set to keep track of visited nodes
visited = set()

# Function for DFS
def dfs(visited, graph, node):
   if node not in visited:
      print(node, end=" ")
      visited.add(node)
      for neighbour in graph[node]:
         dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Path using Depth-First Search:")
dfs(visited, graph, 'A')
```

OUTPUT:
Following is the Path using Depth-First Search:
A B D E F C

AI Practical-2

Name: Sagar Wavhal
Roll No.: 81
Class & Div: TE – B

Title: Implement A star Algorithm for any game search problem Program A* Algorithm.

Program:

```
from collections import deque

class Graph:
    def __init__(self, adjacency_list):
        self.adjacency_list = adjacency_list

    def get_neighbors(self, node):
        return self.adjacency_list.get(node, [])

    # Heuristic function: here all values are equal (can be customized)
    def h(self, node):
        H = {
            'A': 1,
            'B': 1,
            'C': 1,
            'D': 1
        }
        return H.get(node, float('inf'))

    def a_star_algorithm(self, start_node, stop_node):
        open_list = set([start_node])  # Nodes to be evaluated
        closed_list = set()            # Nodes already evaluated

        g = {start_node: 0}            # Cost from start to current node
        parents = {start_node: start_node}  # Parent tracking

        while open_list:
            # Find node in open_list with lowest f(n) = g(n) + h(n)
            n = min(open_list, key=lambda x: g[x] + self.h(x))

            if n == stop_node:
                # Reconstruct path
                path = []
```

```
            while parents[n] != n:
                path.append(n)
                n = parents[n]
            path.append(start_node)
            path.reverse()
            print('Path found:', path)
            return path

        # For all neighbors of current node
        for (m, weight) in self.get_neighbors(n):
            tentative_g = g[n] + weight

            if m not in open_list and m not in closed_list:
                open_list.add(m)
                parents[m] = n
                g[m] = tentative_g
            else:
                if tentative_g < g.get(m, float('inf')):
                    g[m] = tentative_g
                    parents[m] = n
                    if m in closed_list:
                        closed_list.remove(m)
                        open_list.add(m)

        open_list.remove(n)
        closed_list.add(n)

    print('Path does not exist!')
    return None

# Example usage:
adjacency_list = {
   'A': [('B', 1), ('C', 3), ('D', 7)],
   'B': [('D', 5)],
   'C': [('D', 12)]
}
graph = Graph(adjacency_list)
graph.a_star_algorithm('A', 'D')
```

OUTPUT:

Path found: ['A', 'B', 'D']

AI Practical-3

Name: Sagar Wavhal
Roll No.: 81
Class & Div: TE - B

Title: Implement Greedy search algorithm for selection sort Program Greedy search Algorithm for following sort programs:

Program:

```
# Selection Sort in Python
# Time Complexity: O(n^2)

def selectionSort(array, size):
    for i in range(size):
        min_index = i

        # Find the index of the minimum element in the unsorted part
        for j in range(i + 1, size):
            if array[j] < array[min_index]:
                min_index = j

        # Swap the found minimum element with the first element
        array[i], array[min_index] = array[min_index], array[i]

# Driver Code
arr = [-2, 45, 0, 11, -9, 88, -97, -202, 747]
size = len(arr)

selectionSort(arr, size)

print("The array after sorting in Ascending Order by Selection Sort is:")
print(arr)
```

OUTPUT:

```
 The array after sorting in Ascending Order by Selection Sort is:
[-202, -97, -9, -2, 0, 11, 45, 88, 747]
```

AI Practical – 4

Name: Sagar Wavhal
Roll No.: 81
Class & Div: TE - B

Title: Implement a solution for a constraint satisfaction problem using branch and bound and backtracking for n-queens problem or a graph coloring problem
Program: n-queens problem

```
# N-Queens problem using Backtracking
N = 4  # You can change this to solve for any N

def printSolution(board):
    for row in board:
        for val in row:
            print(val, end=" ")
        print()

# Utility function to check if a queen can be safely placed at board[row][col]
def isSafe(board, row, col):
    # Check the current row on the left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check the upper diagonal on the left side
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check the lower diagonal on the left side
    i, j = row, col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True
```

```
# Recursive utility to solve N-Queens problem
def solveNQUtil(board, col):
    # Base case: If all queens are placed
    if col >= N:
        return True

    # Try placing a queen in all rows in this column
    for i in range(N):
        if isSafe(board, i, col):
            board[i][col] = 1  # Place queen

            # Recur to place rest of the queens
            if solveNQUtil(board, col + 1):
                return True

            # If placing queen doesn't lead to a solution, backtrack
            board[i][col] = 0

    return False  # If no place is possible in this column

# Main function to solve N-Queens problem
def solveNQ():
    board = [[0 for _ in range(N)] for _ in range(N)]

    if not solveNQUtil(board, 0):
        print("Solution does not exist")
        return False

    print("One of the solutions to the N-Queens problem is:")
    printSolution(board)
    return True

# Driver Code
solveNQ()
```

OUTPUT:

One of the solutions to the N-Queens problem is:

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

This means:

- Queen is at (0,1), (1,3), (2,0), (3,2)

AI Practical – 5

Name: Sagar Wavhal
Roll No.: 81
Class & Div: TE - B

Title: Develop an elementary chatbot for any suitable customer interaction application

Program : Chatbot Program

```
def greet(bot_name, birth_year):
    print(f"Hello! My name is {bot_name}.")
    print(f"I was created in {birth_year}.")

def remind_name():
    print('Please, remind me your name.')
    name = input()
    print(f"What a great name you have, {name}!")

def guess_age():
    print('Let me guess your age.')
    print('Enter remainders of Class & Class & Class & Dividing your age by 3, 5, and 7.')

    rem3 = int(input())
    rem5 = int(input())
    rem7 = int(input())

    age = (rem3 * 70 + rem5 * 21 + rem7 * 15) % 105
    print(f"Your age is {age}; that's a good time to start programming!")

def count():
    print('Now I will prove to you that I can count to any number you want.')
    num = int(input())

    for i in range(num + 1):
        print(f"{i}!")

def test():
    print("Let's test your programming knowledge.")
    print("Why do we use methods?")
    print("1. To repeat a statement multiple times.")
    print("2. To decompose a program into several small subroutines.")
    print("3. To determine the execution time of a program.")
    print("4. To interrupt the execution of a program.")
```

```
    answer = 2
    while True:
        guess = int(input())
        if guess == answer:
            break
        print("Please, try again.")

    print("Completed, have a nice day!")

def end():
    print("Congratulations, have a nice day!")
    print(". ")
    print("..... ")
    print("..... ")
    input()

# ----- Driver Code -----
greet('TE-Chatbot', '2022')  # Change bot name and year as needed
remind_name()
guess_age()
count()
test()
end()
```

OUTPUT:

```
Hello! My name is TE-Chatbot.
I was created in 2022.
Please, remind me your name.
> Prathamesh
What a great name you have, Prathamesh!
Let me guess your age.
Enter remainders of Class & Class & Class & Dividing your age by 3, 5, and 7.
> 2
> 3
> 2
Your age is 23; that's a good time to start programming!
Now I will prove to you that I can count to any number you want.
> 5
0!
1!
2!
3!
4!
```

5!
Let's test your programming knowledge.
Why do we use methods?
1. To repeat a statement multiple times.
2. To decompose a program into several small subroutines.
3. To determine the execution time of a program.
4. To interrupt the execution of a program.
> 1
Please, try again.
> 3
Please, try again.
> 2
Completed, have a nice day!
Congratulations, have a nice day!
.
. .....
. .....